

Algoritmo de Bellman-Ford

Resumo

O algoritmo de Bellman-Ford é um dos algoritmos utilizados para resolver o problema de caminhos mínimos em grafos. Seu objetivo é encontrar a menor distância entre um vértice de origem e todos os outros vértices do grafo, permitindo arestas com pesos negativos. Além disso, é capaz de detectar ciclos de peso negativo no grafo. Sua principal desvantagem é a complexidade maior em comparação ao algoritmo de Dijkstra.

Implementação em C

Nesta implementação, o grafo é representado por uma matriz de adjacência, onde cada posição `matriz[i][j]` armazena o peso da aresta entre os nós `i` e `j`. Caso não exista conexão entre dois nós, o valor armazenado é `0` ou `INFINITO` (neste caso, utiliza-se um valor simbólico para representar a ausência de aresta).

A função principal do algoritmo recebe três parâmetros:

- `matriz`: matriz de adjacência do grafo;
- `num_nos`: quantidade total de nós (vértices);
- `origem`: nó inicial a partir do qual as distâncias serão calculadas.

A função retorna um valor `double` representando o custo total (soma das menores distâncias).

Parte 1 - Inicialização

Inicialmente, é criado um vetor auxiliar `dist`, onde cada posição `dist[i]` armazena a menor distância encontrada até o momento do vértice `i` em relação ao vértice de origem. Esse vetor é inicializado com valores infinitos para todos os vértices, exceto o de origem, que recebe valor `0`:

```
para i ← 0 até n-1 faça
    dist[i] ← INFINITO
fim-para
```

```
dist[origem] ← 0
```

No código em C, todas as distâncias são inicializadas com `INT_MAX / 2`, representando o infinito. O conceito de infinito, neste caso, é representado por um número suficientemente grande para que qualquer distância real seja menor que ele. O uso da divisão por dois (`/ 2`) evita problemas de overflow durante as somas realizadas no relaxamento das arestas.

Parte 2 - Laço Principal de Relaxamento

Diferente do algoritmo de Dijkstra, o Bellman-Ford não seleciona o vértice de menor distância a cada iteração. Em vez disso, ele repete o processo de relax-

amento de todas as arestas por `num_nos - 1` vezes.

O laço principal é executado `num_nos - 1` vezes:

```
para i ← 1 até n-1 faça
  para u ← 0 até n-1 faça
    para v ← 0 até n-1 faça
      se matriz[u][v] ≠ 0 então
        se dist[u] + matriz[u][v] < dist[v] então
          dist[v] ← dist[u] + matriz[u][v]
        fim-se
      fim-se
    fim-para
  fim-para
fim-para
```

Parte 3 - Cálculo do Custo Total

Assim como na implementação do Dijkstra, somamos todas as menores distâncias encontradas:

```
custo ← 0

para i ← 0 até num_nos - 1 faça
  se dist[i] != INFINITO então
    custo ← custo + dist[i]
  fim-se
fim-para
```

Parte 4 - Finalização

Após o laço de relaxamento, o custo total é calculado somando todas as menores distâncias encontradas. Se alguma distância permanecer como infinito, indica que o vértice é inalcançável a partir da origem. O laço final soma todas as distâncias, garantindo que apenas valores alcançáveis sejam incluídos na soma total.

```
free(dist);
return custo;
```

Complexidade

A implementação possui complexidade $O(V^3)$, pois utiliza uma matriz de adjacência e três loops aninhados: um loop externo que executa $V - 1$ vezes e dois loops internos que percorrem todos os vértices, totalizando $O(V \times V \times V) = O(V^3)$.

Para grafos grandes e esparsos, uma implementação utilizando lista de arestas pode reduzir a complexidade para $O(V \times E)$, sendo V o número de vértices e E

o número de arestas.

Comparação com Dijkstra

Critério	Dijkstra	Bellman-Ford
Pesos negativos	Não permite	Permite
Complexidade (matriz)	$O(V^2)$	$O(V^3)$
Complexidade (lista)	$O((V + E) \log V)$	$O(V \times E)$
Eficiência para grafos sem pesos negativos	Mais eficiente	Menos eficiente